

Predicting Football Transfer Fees

GRA41574 – (Big) Data Curation Pipelines and Management

9. December 2025

Student ID's: 1118710, 1118476

GitHub Repository:

<https://github.com/1118476/gra41574-project/tree/main>

1. Introduction

Football is the world's sport.

It makes people cheer, celebrate and cry.

But it also makes them spend money. This makes football a global business with clubs heavily investing into players. According to the Deloitte football money league 2025 reports the twenty teams included, which are all among the biggest clubs in the world, generated a combined revenue of 11.2 Billion US dollar (Deloitte, 2025). Last season the 8 leagues relevant to this project spent a total of 7.9 Billion dollars on new players (Transfermarkt, n.d.). With these numbers spent on players it is crucial to make the right decision when buying a player. This report shows an ML approach to predicting transfer fees in football based on individual player data. Ante (2019) analyzed 389 transfers from the 2018/19 summer transfer window to identify determinants of transfer fees using linear regression with stepwise elimination, showing that factors such as age, height, performance, popularity, and league destination significantly shape player valuation. Idris and Ng (2025) built a machine-learning model to predict football players' market value using a Kaggle dataset from 2017–2020, applying various ML models with hyperparameter tuning to identify the best-performing model and key features by position. Their approach focuses on season-end market values, a limited set of real-match statistics, and a relatively small, pre-structured dataset. In the project presented in this report, a vast dataset spanning many leagues and seasons together with nonlinear ML algorithms is used to predict transfer fees directly combining the use of Idris and Ng (2025) with the use of dependent variable from Ante (2019). It uniquely stands out in the amount of data, both on the transfer as well as on the player stats side as well as with the detailed capturing of individual player performance via stats while not neglecting potential driving factor offside the pitch like age or contract length. The goal of this report is to predict how well individual player data predicts transfer fees in football and can be viewed as a tool for a club looking to sell a player to strengthen their bargaining position. It will be examined what the impact of changing the timeframe to look back on is. Implicitely this report gives a hint to the importance of negotiation and individual scouting when deciding transfer fees.

2. Methods

2.1 Data Description

For the analysis player match data from fbref.com is combined with transfer data from transfermarkt.com. This is done for the top five european leagues as well as the Eredivise, Liga NOS and Championship. Player match data consists of gameplay metrics like goals, assists, pass completion and many more. In total there are 65 metrics from gameplay used

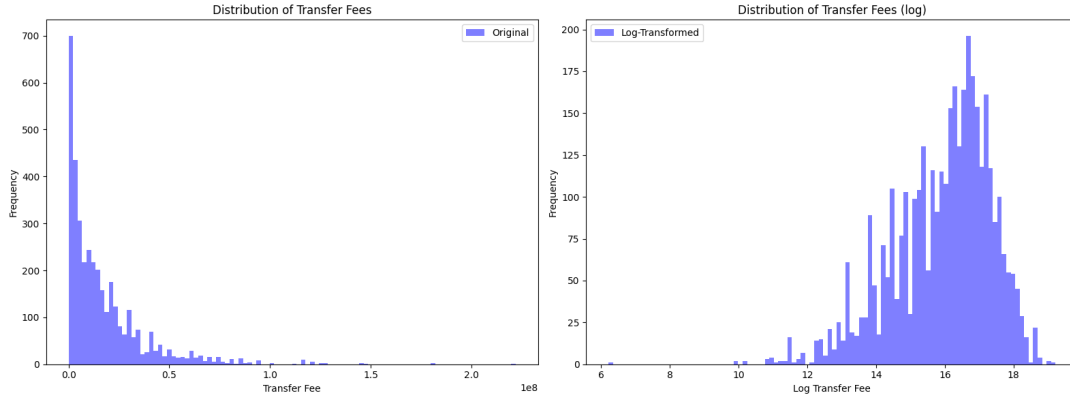


Figure 1: Distribution of transfer fee and logarithm of transfer fee, free transfers excluded

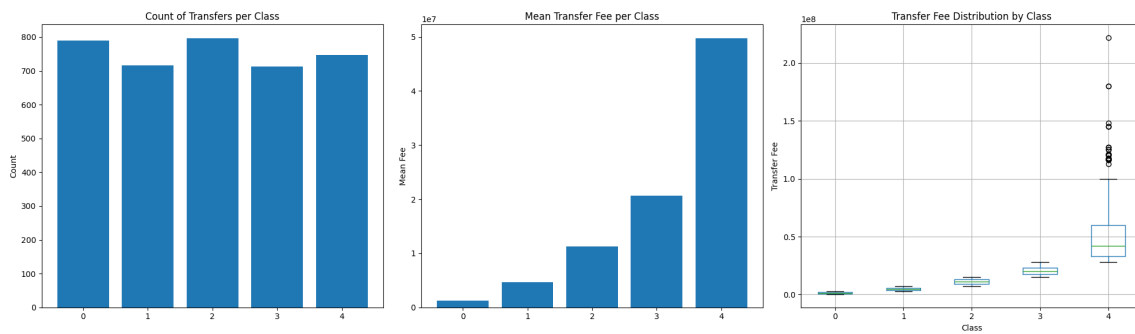


Figure 2: Transfer classes, no. of transfers per class, mean transfer value and boxplot, Class boundaries are set by quantiles: 0, 0.2, 0.4, 0.6, 0.8, 1

for prediction. Note that the scraped data is not the dataframe used for ML. Transfer data brings in metrics from off the field like age, remaining contract length and market value. For the timeframe relevant here, that is from the summer transfer window 2017 to the summer transfer window 2025, the data holds 3762 transfers. That excludes free transfers and loans. The timeframe is constrained by the introduction of detailed data capturing in european top football.

Visualizations Figure 1 shows the distribution of transfer fees and paints a clear picture. The fees are highly skewed due to the overwhelming majority of players being traded for relatively low fees while some are traded for extraordinary amounts. Due to the skewness, the models predict the log of transfer fees in this project (right side of 1). Figure 2 gives information on the dependent variable used for classification. Each bin holds transfer fees in a certain quantile, which guarantees evenly large fee classes. The boxplot charts show that this leads to a finer grained distinction in lower transfer fee regions and rather aggregates fees more broadly in larger fee regions. It can be argued that this is useful for real life situations because it is more crucial for clubs that sell players in lower transfer regions to accurately estimate the fee they can get. In higher transfer fee regions absolute deviations from transfer fees get relatively small and therefore less important.

In other words, it might be reasonable to assume that a small club fights to get 3 million instead of 2.5 for a player, while a larger club might be almost indifferent between selling for 60 or 60.5 million.

Data Scraping and cleaning This paragraph serves to give a quick look into the scraping part, which made up a great chunk of the work put into this project. It is supposed to emphasize the data collection process and shows the dynamic construction of the machine learning input dataframe.

To scrape fbref.com `stats_scraping.py` initializes an object of the class `FBREFScraper` from `fbrefscraper.py` for every combination of season and league. `FBREFScraper` constructs the link to the league and season website and extracts the link leading to the stats of a single game from a table holding all season games. It then iterates over these links, scraping data from the 12 tables, 6 for the home and 6 for the away team, that contain stats from the players and are then concatenated appropriately into one. The output is one csv file for a single season of a league. The final dataframe holding all scraped individual player data has 662867 entries. From these csv files the main dataframe holding all the player data is constructed in `build_player_data_csv.py` and an index column is constructed signaling the season half in which the stats in a row are from. The index ranges from 0 (First half of 2017/2018) to 15 (Second half of 2024/2025). NaN values are handled by replacing them with median values. This allows models that cannot handle missing values to train on a complete dataset without artificially shifting the feature distribution as much as a mean replacement would.

To get transfer infos from transfermarkt.com, each season of each league was iterated over. For each player a webpage for each of his transfers was loaded from which information regarding this particular transfer was scraped. For every transfer a parser object from `transfers_parser.py` was called and, if successful, the parsed data on name, market value, selling club, transfer fee and more was written into csv files in a clean standardized format. These csv files are then read into a dataframe in `build_transfer_data_csv.py` where it is prepared to be processed into ML ready format. First the year and month of the transfers is extracted and a column "transfer_window_idx" is constructed, ranging from 0 for Summer 2017 to 16 for Summer 2025. Only the transfers from the eight focused leagues are kept and loan transfers are excluded, since the paid fees are a lot less than for real transfers they are distorted for the objective of predicting actual transfer fees. Free transfers happen only when contracts expire and are therefore excluded since they can be explained solely through contract length.

Data Preprocessing for ML From raw scraped data there are still steps needed to be done to get it ready for ML analysis. The final dataframe used for ML consists of aggregated data for all traded players in a specific timeframe. This timeframe ends

at the transfer window in which a player was traded and looks back a certain amount of season halves. The construction of this dataframe is dynamically handled by the `DataProcessor` class from `dataprocessor.py`. In `run.py` it gets passed the full player data, full transfer data, as well as an optional argument specifying the timeframe in terms of season halves to look back on. From `player_data.csv` the stats were sliced out depending on the relevant season halves that are dated back from the time of the transfer. In the constructor the main function of the `dataprocessor` is called, that is `loop_transfers()`. This function iterates over all rows in `transfer_data` and for every transfer executes `find_main_position()` to determine the main position of the transferred player, `aggregate_data()` to sum or average the player stats from the relevant timeframe into one single row and `append_data()` to concatenate transfer specific data and player stats in one row and then append this row to the ML dataframe holding the data from previously looped transfers. Correlation between features is checked in the train set to assess multicollinearity, reduce standard errors, make possible for the model to isolate features' individual effects, and reduce the risk of overfitting by preventing the model from learning from noise. Several correlation values were checked to find the optimal one (> 0.8 and > 0.9). Since `catboost` can be passed categorical columns (like `selling_club`), it gets its own separate dataframe passed the same way, via `@property` decorated functions in `DataProcessor()`. For classification models `'fee_class'` column is added (figure 2), with 5 classes balanced in size. At the same time, `'transfer_fee'` column is dropped from this dataset to avoid perfect correlation between this feature and the dependent variable. The final size of the dataframe used for training the models depends on the number of relevant season halves, for one it is (2124, 65).

2.2 ML models design

Classification and Regression models The following models were used to implement classification: Logistic Regression, Random Forest Classifier, XGBoost Classifier, and Gradient Boosting Classifier with `"fee_class"` as the dependent variable. Initially, each of these models is trained on default hyperparameters, which allows for consistent comparison based on their predicting quality and helps to choose the best model for this specific data and goal. The following metrics were used for model evaluation: accuracy, precision, recall, and F1 score. Logistic regression is a type of generalized linear model with the ordinal context of classification in this case for multiple ordered classes. It is used as a simple baseline model for comparison to evaluate how much nonlinearity is needed. Similar set of regression models was evaluated: Linear Regression, three tree-based regressions - Random Forest, XGBoost, Gradient Boosting, and CatBoost, which is also a Decision Tree based algorithm, with `"transfer_fee"` as dependent variable. Again, since linear regression just models the target variable as linear combination of independent variables, it probably will be too simple in the case with our data, so it is added for com-

parison. Random forest is an ensemble method of many independent decision trees, where predictions are aggregated to reduce variance, gradient boosting build trees sequentially with each next tree correcting errors of the previous, XGBoost is an optimized gradient boosting model with regularization, and CatBoost is a gradient boosting algorithm with built-in categorical encoding (Fatima et al., 2023). Equivalently to the classification case, models were first trained with default hyperparameters. MAE, MSE, R^2 metrics were used for estimation of regressions quality. It is reasonable to expect CatBoost to be the best regression model, since it uses Ordered Target Encoding to avoid leakage.

Encoding Along with numerical data, the dataset has crucial categorical data (position, selling league, and selling club), which must be converted to numerical to be used by the majority of classification/regression models. For this purpose, OneHotEncoder and LabelEncoder are used. OneHot creates a dummy variable for each unique value of a categorical variable, while Label Encoder transforms categorical data by assigning each unique value its own "class". The whole project pipeline supports both of the encoders, which can be changed by explicitly sending the `--encoder` parameter when running the project. Each of them is evaluated separately, and the differences are explained in the results section.

Tuning After the evaluation of baseline models, 3 with the best metrics were chosen for further estimation: XGBoost for both classification and regression and CatBoost for regression. RandomizedSearchCV was used for model tuning. This method samples random sets of hyperparameters from given distributions and fits and evaluates the model using cross-validation, choosing the best parameters. The decision to use randomized search over grid search was supported by the fact that the first works a lot faster with large sets of data and is more efficient when hyperparameter space is big.

3. Results

Figure 3b illustrates a feature of the given football data that the ML models will have to handle: the are not always linear relationships. The most obvious case is depicted in 3b where the transfer fees are plotted based on the age of the player. It is in line with real world expectations to see the highest fees being payed for players in their mid twenties, little before the prime age of a footballer. A more detailed visualization of metrics from a ML dataframe can be found in `visualization.ipynb`.

Figure 3a depicts the median transfer fee of transfer from a league showing a clear difference between leagues. This was constructed using `transfer_data.groupby('selling_league')['transfer_fee'].median()`. Due to this characteristic the categorical variable "selling_league" is included in the dataset. It is worth noting that looking at buying league would yield a similar pattern, however, from

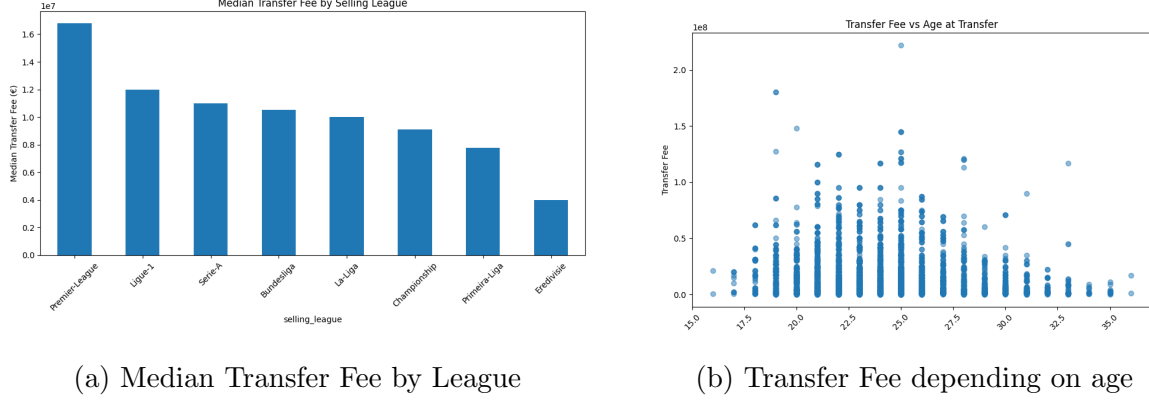


Figure 3

a perspective of real world applicability, the dataset only holds metrics that could be observed by a club official wanting to estimate the fee of a potential player sell.

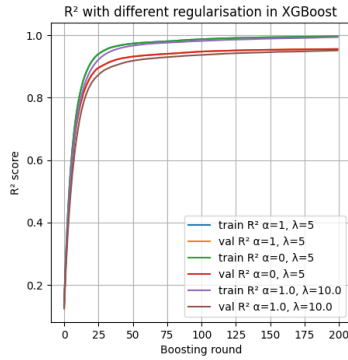
Table 1 presents a summary of the quality metrics of certain models with different types of encoding.¹ Logistic and linear regressions have the poorest performance, indicating, that they are too simple to capture all dependencies and effects. However, they overfit the least. The reason is that our data might have various non-linear patterns and feature interactions that simple model can't capture (see figure 3b), and categorical data makes linear approaches worse. The other three (decision tree based) models show similar better performance, because all three are ensemble methods based on the decision trees that have the same logic but differ in the way they are combined and constructed. XGBoost has the best quality scores out of all three models, which is why it is chosen as the main model for both classification and regression. Even though it overfits a lot, indicating too complex model that tends to "memorize" patterns instead of actually finding dependencies between variables, regularization parameters are tuned in randomized search, reducing overfitting. Changing the encoder to one-hot decreases quality metrics for random forest and gradient boosting models, while slightly enhancing performance of XGBoost and linear models for both classification and regression (catboost results stay the same, since encoding doesn't influence it). This type of encoding greatly expands data, making it hard for ordinary tree models to find the best splits. On the other hand, linear models win from one-hot encoder because in the case with label encoders, they try to assume an order between values (e.g. 'one league' < 'another league') in categorical variables, which doesn't exist in reality. However, as expected, CatBoost model has the best quality metrics (even though not very different from XGBoost with encoder), which can be explained by the fact that this model can handle high-cardinality categorical variables without introducing the drawbacks of one-hot/label encoders. While tuning models, following hyper-

¹Due to the limited report size, we are not including all the models results (baseline/tuned), but it is possible to view them in the log files on GitHub (`model_run_output_label.log`, `model_run_output_onehot.log`)

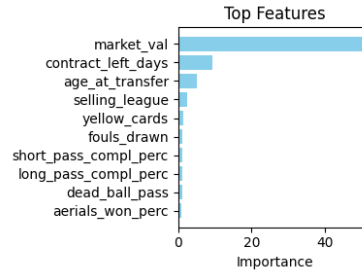
parameters were found: 'XGB classifier': `n_estimators = 800`, `learning_rate = 0.155`, `reg_alpha = 0`, `reg_lambda = 1`, 'XGB regressor': `n_estimators = 700`, `learning_rate = 0.1`, `reg_alpha = 0`, `reg_lambda = 5`, 'Cat Boost': `iterations = 1000`, `learning_rate = 0.2`, `l2_leaf_reg = 30`. Figure 5 (a) illustrates how overfitting issue can be addressed by changing regularization parameters.

Table 1: Models Comparison: Accuracy for classification, R^2 for regression

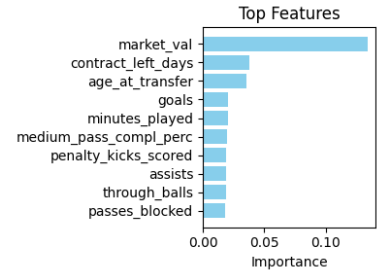
Model	Label Encoding		One-Hot Encoding	
	Train	Test	Train	Test
Logistic Regression	0.65	0.59	0.77	0.63
XGBoost Classifier (tuned)	1.00	0.81	1.00	0.82
Linear Regression	0.60	0.59	0.69	0.59
XGBoost Regressor (tuned)	0.97	0.87	0.98	0.88
CatBoost Regressor (tuned)	0.99	0.89	0.99	0.89



(a) Regularization in XGBoost



(b) Regression



(c) Classification

Figures 4b and 4c describe 10 most important features in classification and regression. For regression, transfer fees relies heavily on market value of the player, his age, contract time left, league he plays in, yellow cards and some other stats. Similar patterns are observed for classification with goals having higher importance. This observation validates the model performance and provides valuable insights for decision-making in football transfers that align almost perfectly with real-life scenarios. It was also examined how model performance changes when using different timeframes (player data across several season halves), but no significant differences in results were observed.

4. Conclusion

This project demonstrates that football transfer fees can be predicted with adequate accuracy based on the individual player statistics data and context (league, age of the player, etc.). XGBoost and CatBoost models perform a lot better than linear models or simpler tree ensemble models, indicating complex, nonlinear relationships in data. Biggest drivers

of transfer fees is the combination of both on-field (goals, passes, etc) and off-field (market value, league, club, age, contract length) data. A manager looking to sell a player can use the market value metric from transfermarkt.com as a strong indicator. Additionally the models emphasize the increase in bargaining power given by a players age and by his contract length. Individual player stats play a relatively small role, but are still a valid argument in a negotiation. However, there still exists an overfitting issue that makes predictions less reliable, so the prediction model should be used only as an addition to professional scouts work and not as an individual tool. Nevertheless, lack of accuracy and overfit is fully in line with real-life scenario: transfer fees can be very subjective assessments that includes player’s reputation and popularity, agent’s biased impressions, football market, and other data, which can not be tracked and used for prediction. That is, this research implicitly shows and emphasizes variability and unpredictability in transfer fees.

This project has other limitations, such as usage of only 8 biggest leagues, that can introduce sample bias and threaten external validity and not accounting for market dynamics. Moreover, exclusion of free transfers biases distribution upwards and can lead to a less representative sample. Also, lack of the data on high transfer fees weakens the model and its applicability for the large values in the market. Overall, the models provide a strong foundation for future research that can incorporate additional contextual variables like reputation (can be proxied by instagram followers), coach, or league difficulty or expand data to more leagues.

References

- Ante, L. (2019). Determinants of transfer fees: Evidence from the five major european football leagues. <https://doi.org/10.13140/RG.2.2.18356.91526/1>
- Deloitte. (2025, January). *Deloitte football money league 2025*. Deloitte United Kingdom. <https://www.deloitte.com/uk/en/services/financial-advisory/analysis/deloitte-football-money-league.html>
- Fatima, S., Hussain, A., Amir, S., Ahmed, S. H., & Aslam, S. (2023). Xgboost and random forest algorithms: An in-depth analysis. *Pakistan Journal of Scientific Research*, 3(1), 26–31. <https://doi.org/10.57041/pjosr.v3i1.946>
- Idris, M., & Ng, S. (2025). Developing a predictive model for football players’ market value using machine learning. *Journal of Informatics and Web Engineering*, 4(3), 203–214. <https://doi.org/10.33093/jiwe.2025.4.3.12>
- Transfermarkt. (n.d.). *Transfer balance statistics* [Accessed: December 9, 2025]. <https://www.transfermarkt.com/statistik/transfersalden>